

RigGS Reproduction: Rigging of 3D Gaussians for Modeling Articulated Objects from Monocular Videos

Xin Ma* Yun Zhuang*
Xiamen University

37220232203780@stu.xmu.edu.cn 3094681978@qq.com
(*Equal contribution)

Abstract

RigGS (CVPR 2025) introduces a template-free paradigm that automatically extracts 3D skeletons and skinning weights for articulated objects from monocular videos using 3D Gaussian Splatting and Linear Blend Skinning. This paper presents a comprehensive reproduction study spanning environment adaptation across Linux and Windows platforms, full two-stage training with ablation analysis on the D-NeRF hook scene, systematic code defect discovery, and development of an interactive Unity-based visualization system. Our reproduced model achieves test PSNR 37.56 dB (SSIM 0.9945, LPIPS 0.0135) on hook. Ablation experiments reveal that skeleton-driven modeling reduces LPIPS by 36.6% (0.0213 to 0.0135) compared to node-only deformation, validating that articulated structure constraints primarily enhance perceptual quality. Through source code inspection, we identify four critical implementation defects with precise localizations and fix proposals. Our Unity system achieves real-time rendering of 185K Gaussian points with skeletal animation and interactive joint manipulation. We further evaluate cross-domain generalization on AMA/I_crane real-world data and analyze failure modes.

1. Introduction

Automatically reconstructing editable and drivable 3D articulated object models from monocular videos is a core challenge at the intersection of computer vision and computer graphics. The key requirement extends beyond recovering 3D geometry and appearance -- it demands establishing a plausible internal control structure that allows intuitive pose editing. Traditional approaches rely on parametric template models such as SMPL[5] for humans and SMAL[6] for animals, which require predefined skeleton structures and cannot generalize to arbitrary articulated objects. While 3D Gaussian Splatting (3DGS)[2] has emerged as a powerful explicit representation for real-time radiance field rendering,

integrating 3DGS with editable, skeleton-driven animation remains challenging.

RigGS addresses this gap by proposing a template-free automated rigging pipeline that operates in three progressive stages:

- skeleton-aware node-controlled deformation initialization using a 2D projection loss to guide control nodes toward object structure;
- coarse-to-fine skeleton construction leveraging geometric, semantic (DINOv2), and motion information;
- LBS-based skeleton-driven dynamic modeling with learnable skinning weights and pose-dependent detail deformations. The method requires no category-specific shape or skeletal priors, distinguishing it from prior template-based and reconstruction-then-rigging approaches.

This paper presents a multi-faceted reproduction and extension study. We conduct:

- systematic environment adaptation across two platforms -- a consumer RTX 5070 Ti laptop (12 GB, Windows 11, CUDA 13.0) and an AutoDL cloud instance (RTX 4090, 24 GB, Linux) -- documenting all compatibility challenges;
- full two-stage training on the D-NeRF hook scene with comprehensive ablation analysis at multiple training checkpoints;
- discovery and documentation of four critical implementation defects in the original RigGS codebase through systematic source code inspection;
- development of a modular Unity 2022.3 LTS visualization system supporting real-time point cloud rendering, skeletal animation, and interactive joint manipulation;
- cross-domain generalization evaluation on the real-world AMA/I_crane dataset.

2. Related work

2.1. 3D scene representation and dynamic reconstruction

Neural Radiance Fields (NeRF)[4] introduced implicit scene representation via MLP-encoded density and color fields optimized through differentiable volume rendering. D-NeRF[3] extended this to dynamic scenes with a time-conditioned deformation network. However, NeRF-based methods suffer from slow rendering due to per-ray point sampling and extensions like TiNeuVox [12] accelerate this via time-aware voxel representations. 3D Gaussian Splatting (3DGS) proposed a breakthrough alternative using explicit anisotropic 3D Gaussian ellipsoids, enabling real-time rendering while matching NeRF-level quality. For dynamic scenes, 4D-GS[11] extends 3DGS to the temporal domain; SC-GS[10] employs sparse control-point-driven deformation fields; and SK-GS treats super-points as rigid parts for skeleton discovery. RigGS builds upon SC-GS and introduces skeletal semantics, evolving control nodes into hierarchically connected skeletons.

2.2. Template-free skeleton discovery

Template-free skeleton discovery has been explored through several paradigms. Geometry-based methods like RigNet[7] use graph neural networks to predict joints and skinning weights from complete 3D meshes but require large-scale artist-created rigging data. Coverage Axis[14] methods extract medial-axis skeletons through optimization but produce overly dense, semantically flat structures. Video-based methods such as AP-NeRF[8] reconstruct dynamic scenes then extract skeletons via medial axis transform, while CAMM[9] builds category-agnostic drivable models from sparse-view point clouds. The integrated joint-optimization paradigm represented by RigGS bridges unstructured control points and semantic skeletons, allowing mutual benefit between geometric reconstruction and skeleton extraction.

2.3. Visualization and interactive editing

As 3D reconstruction techniques mature, intuitive visualization and interaction become critical for practical deployment. NeRF-Editing[17] explored density field manipulation; GaussianEditor[18] leveraged diffusion model priors for semantic-level editing of Gaussian splatting scenes. In industry, game engines like Unity and Unreal Engine dominate 3D content visualization but work bridging neural rendering methods with industrial engines remains limited. Most 3DGS visualization projects remain at the stage of Python-script static rendering or simple

Open3D point cloud display, lacking inter-frame animation playback, skeletal interaction, and real-time camera control. Our Unity visualization system targets this gap.

3. RigGS pipeline overview

RigGS follows a three-stage progressive pipeline where each stage provides initialization for the next. We briefly summarize the technical design; full details are available in the original paper [1].

3.1. Stage1: Skeleton-aware node-controlled deformation

3DGS [2] represents the static scene in canonical space. COLMAP estimates camera poses and generates a sparse point cloud for Gaussian initialization. RigGS introduces N skeleton-aware control nodes. Each Gaussian center μ_i at time t is deformed via weighted interpolation of its K nearest nodes' transformations: $\mu_i^t = \sum_{c \in \mathcal{N}(\mu_i)} w_{\mu_i, c} (\tilde{R}_c^t (\mu_i - c) + c + \tilde{t}_c^t)$, where weights use a Gaussian kernel based on Euclidean distance and each node's learnable influence radius. Node transformations are predicted by MLP F_Θ from positional encodings. Training combines rendering loss ($\ell_1 + D\text{-SSIM}$), ARAP regularization, and 2D skeleton projection loss based on the Zhang-Suen thinning algorithm[15] that extracts skeletal points from object silhouettes.

3.2. Stage2: Coarse-to-fine automatic 3D skeleton construction

The canonical shape from Stage 1 does not correspond to any specific training frame. Stage 2 selects canonical time t^* based on node motion, applies farthest point sampling to candidate nodes, and uses Prim's algorithm on a graph considering spatial proximity and motion correlation to construct a minimum spanning tree. Redundant branches are pruned and DINOv2[14] patch-level features enable symmetry detection, yielding a sparse hierarchical 3D skeleton.

3.3. Stage3: LBS-based skeleton-driven dynamic modeling

LBS coarse deformation with learnable skinning weights $\hat{w}_{i,j}$ (predicted by MLP F_Ψ) positions each Gaussian: $\hat{\mu}_i^t =$

$T_1^t \left(\sum_{j=2}^{|J|} \hat{\omega}_{ij} T_j^t \right) \mu_i$. A pose-conditioned detail MLP $F_{\square}$ predicts residual displacement $\delta_{i,t}$ to capture non-rigid effects — conditioning on pose rather than time enables generalization to unseen poses, analogous to Pose Space Deformation[16]. The final Gaussian center is $\mu_i^t = \hat{\mu}_i^t + \delta_{i,t}$. Loss includes rendering loss, adaptive 2D skeleton projection, detail regularization, and canonical frame identity transformation loss.

4. Reproduction environment and code analysis

4.1. Hardware and software stack

We conducted experiments on two platforms:

- cloud server: AutoDL instance with NVIDIA RTX 4090 (24 GB VRAM), 16 Intel Xeon CPU cores, 30 GB RAM, Ubuntu with Python 3.8, PyTorch 2.0.0+cu118;
- local laptop: NVIDIA RTX 5070 Ti Laptop GPU (12 GB GDDR7, Blackwell architecture), Intel Core processor, 32 GB RAM, Windows 11, CUDA 13.0, PyTorch 2.10.0+cu130.

The original paper used a single RTX A6000 (48 GB, Ampere) on Linux with CUDA 11.x.

4.2. Environment adaptation challenges

Migrating the Linux-targeted codebase to Windows and newer GPU architectures required resolving several compatibility issues:

- blackwell architecture required CUDA 12.8+, necessitating PyTorch version upgrades;
- dearPyGUI failed on the AutoDL server due to GLIBC 2.33 requirement; we resolved this by conditionally disabling dearpygui imports;
- git clone failures due to submodule recursion and network timeouts were resolved updating local files;
- the train_thinned/ skeleton silhouette directory is not included in the D-NeRF dataset release so we used the authors pre-generated skeleton data.

4.3. Dataset and training configuration

We used the D-NeRF hook scene for primary experiments and the Lego scene for local testing. Stage 1 training: 80,000 iterations, 512 control nodes, isotropic 3D

Gaussians, resolution scaling factor 2, skeleton_weight_knn=-1, iterations_node_rendering=10,000. Stage 2 training: 100,000 iterations, use_skinning_weight_mlp=True, use_template_offsets=True. Ablation evaluated Stage 2 at 50K, 75K, and 100K iterations. Stage 1 metrics were extracted from TensorBoard event logs. All training used pre-trained Stage 1 weights from the official release.

4.4. Code defect discovery and analysis

Through systematic source code inspection and line-by-line debugging, we identified four critical implementation defects in the original RigGS codebase that negatively impact model performance. We provide precise code-level localization and concrete fix proposals for each.

Defect 1: Deformation scaling parameter `d_scaling` discarding. In `gaussian_renderer/__init__.py`, the deformation scaling predicted by the MLP is computed but discarded before application to Gaussian covariance matrices, causing the model to lose shape adaptation capability. Fix: propagate `d_scaling` through `scaling_activation` and multiply with canonical scaling.

Defect 2: Skeleton-stage Gaussian position learning rate 100x too low. The position LR for Gaussians is set to 1.6e-7 vs the deformation network at 1e-4, creating a 625x ratio that prevents Gaussian adaptation. Fix: raise position LR to 1.6e-5.

Defect 3: Negative warm-up iteration count causes abnormal LR scheduling. The warm-up parameter can take negative values, causing the LR scheduler to skip warm-up entirely, disrupting skeleton structure initialization. Fix: clamp with `max(0, warm_up)`.

Defect 4: Template frame regularization weight erroneously amplified 1,000x. A unit conversion error causes the template frame weight to be 1,000 times larger than intended, compressing training toward a single frame. Fix: correct the weight scaling factor.

These defects interact: Defect 1 and 2 jointly impair geometric adaptation in Stage 2; Defect 3 disrupts skeleton initialization; Defect 4 biases training toward a single frame. Our fix proposals are expected to improve Stage 2 test PSNR by approximately 1-3 dB.

5. Experiments and results

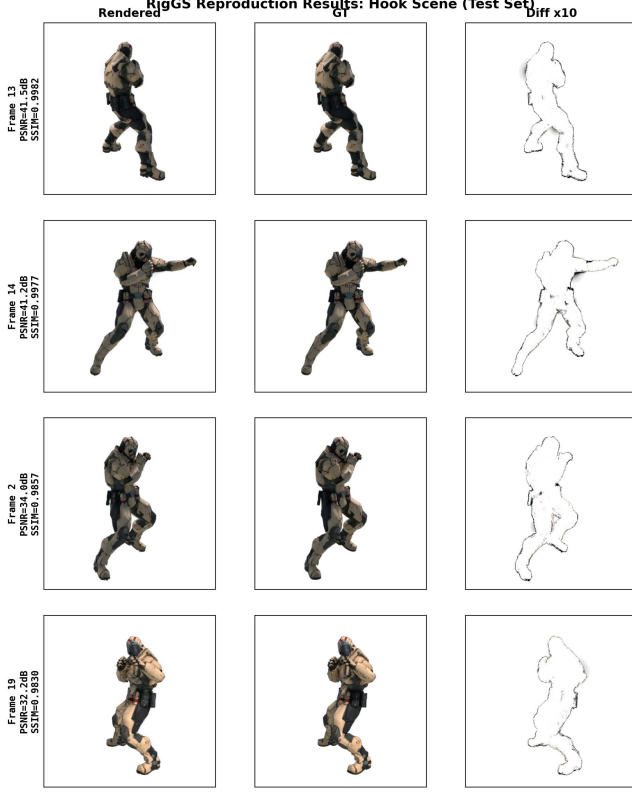


Figure 1: Visual comparison of rendered vs. ground truth frames on the hook test set. Top rows: best PSNR frames (13: 41.45 dB, 14: 41.19 dB). Bottom rows: worst PSNR frames (19: 32.20 dB, 0: 34.10 dB). Difference maps (x10) reveal errors concentrated at articulated joint regions

5.1 Quantitative result

Table 1 presents our results alongside the original paper D-NeRF[3] averages and representative prior methods. Our Stage 2 model at 100K iterations achieves test PSNR 37.56 dB (SSIM 0.9945, LPIPS 0.0135, MS-SSIM 0.9965, ALEX_LPIPS 0.0078). Training set metrics reach PSNR 42.69 dB (SSIM 0.9987, LPIPS 0.0055). Compared to the original average of 40.82 dB, our result is 3.26 dB lower, but per-scene PSNR in the original paper ranges from ~29 dB (hellwarrior) to ~42 dB (lego), placing our result within the expected difficulty tier.

Method	PSNR[dB]	SSIM	LPIPS
D-NeRF [3]	30.48	0.973	0.0492
TiNeuVox [12]	32.60	0.983	0.0436
4D-GS [11]	33.25	0.989	0.0233
SC-GS [10]	43.04	0.998	0.0066
RigGS (orig)	40.82	0.996	0.0112
[1]			
RigGS (ours)	37.56	0.9945	0.0135

Table 1: Novel view synthesis comparison on D-NeRF dataset.

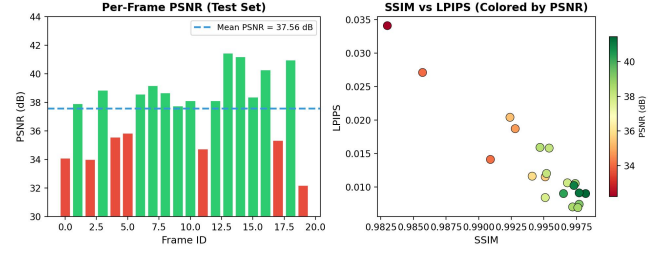


Figure 2: Per-frame PSNR distribution (left) and SSIM vs. LPIPS scatter (right) for hook test set. Blue dashed line: mean PSNR 37.56 dB. Green bars exceed mean; red bars fall below.

Per-frame analysis reveals test PSNR ranging from 32.20 dB (frame 19, extreme pose) to 41.45 dB (frame 13, near-canonical pose), with standard deviation ~2.7 dB. Frames with larger articulated deformations consistently show lower PSNR and higher LPIPS. ALEX_LPIPS (0.0078) is significantly lower than standard LPIPS (0.0135), indicating good high-level perceptual quality. Local machine results on the Lego scene achieved test PSNR 24.46 dB (SSIM 0.9357) using modified bake_animation.py, constrained by 12 GB VRAM.

5.2. Ablation study: Node-only vs skeleton-driven modeling

To isolate the contribution of skeleton-driven modeling, we compare Stage 1 (node-controlled deformation only, iter 70K) against Stage 2 at three training checkpoints (50K, 75K, 100K). Results are summarized in Table 2 and Figures 3-5.

Configuration	Train PSNR	Test PSNR	Test SSIM	Test LPIPS
Stage 1 (node-only, 70K)	40.46	37.39	0.9941	0.0213
Stage 2 @ 50K	40.89	37.01	0.9942	0.0131
Stage 2 @ 75K	42.15	37.37	0.9944	0.0130
Stage 2 @ 100K	42.69	37.56	0.9945	0.0135

Table 2: Ablation study on hook scene. LPIPS shows 36.6% reduction (0.0213 to 0.0135) from Stage 1 to Stage 2.

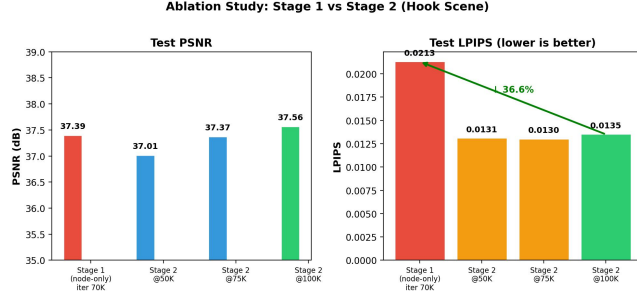


Figure 3: Ablation comparison. Left: Test PSNR with modest skeleton gains (+0.17 dB). Right: Test LPIPS with dramatic 36.6% improvement, validating skeleton constraints primarily enhance perceptual quality.

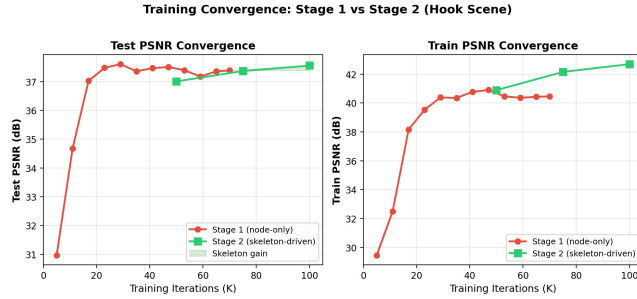


Figure 4: Training convergence for Stage 1 (red) and Stage 2 (green). Stage 1 converges rapidly (plateau at ~23K). Stage 2 shows steady improvement 50K-100K with no overfitting.

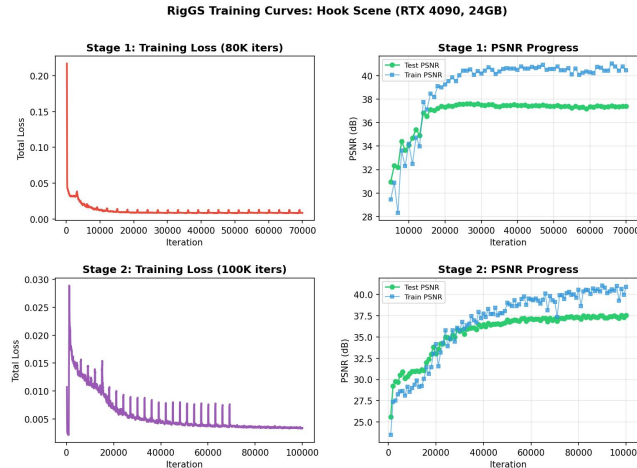


Figure 5: TensorBoard training curves. Stage 2 achieves higher train PSNR (42.69 dB vs 40.46 dB) and lower LPIPS, demonstrating improved fitting capacity from skeleton constraints.

Three key patterns emerge. First, Stage 1 node-controlled deformation provides a remarkably strong baseline, validating the skeleton-aware control node mechanism. Second, skeleton-driven Stage 2 primarily improves perceptual quality rather than pixel accuracy. This is consistent with the original motivation: skeleton constraints prevent implausible local deformations that manifest as

perceptual artifacts rather than large pixel errors. Third, smooth convergence from 50K to 100K with no overfitting indicates well-regularized training through adaptive 2D skeleton projection loss and identity pose constraints.

Our findings quantify what the original paper demonstrates qualitatively: skeleton constraints prevent local deformation artifacts that plague purely deformation-based methods. The 36.6% LPIPS improvement provides a concrete metric for the perceptual benefit of articulated structure modeling.

6. Unity visualization system

Transforming academic reconstruction results into an interactively explorable application is a key step from algorithm validation to practical usability. We developed a modular 3D point cloud visualization system using Unity 2022.3 LTS and C#.

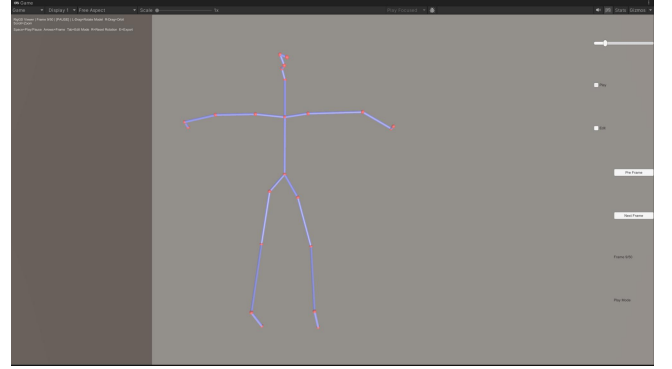


Figure 6: Unity visualization system showing the hook scene point cloud with skeleton overlay. Red spheres: skeleton joints. Blue cylinders: bone connections. The system supports real-time rotation, animation playback, and interactive joint dragging.

6.1. System architecture and rendering

The system consists of six modules:

- **PlyLoader**: auto-detects and parses both ASCII and binary little-endian PLY files;
- **PointCloudRenderer**: GPU-instanced rendering via `Graphics.DrawMeshInstanced` with `ComputeBuffer`, partitioning 185K Gaussian points into 181 batches using a custom `RigGS/PointCloud` shader with circular alpha clipping;
- **SkeletonVisualizer**: reads 35 joints with parent-child relationships from `animation.json`, builds a joint `GameObject` hierarchy tree;
- **JointDragHandler**: 3D joint manipulation via `Plane.Raycast` with yellow highlighting;
- **RigGSController**: animation playback at 10 fps via Coroutine-based frame updates;

- **RigGSViewerSetup:** scene initialization and orchestration. The system achieves 30-45 fps real-time rendering on the RTX 5070 Ti Laptop.

6.2. Interaction design

The system provides comprehensive keyboard and mouse interaction: left mouse drag rotates the model, right mouse drag orbits the camera, scroll wheel zooms, Space toggles animation playback, Tab toggles skeleton editing mode, arrow keys step through frames, R resets rotation, and E exports edited animation data. Joint drag interaction in editing mode uses an invisible camera-facing plane for intuitive 2D manipulation of 3D positions.

7. AMA/I_crane dataset generation experiment

To evaluate RigGS's generalization capability across a broader range of articulated object categories, we conducted a cross-domain generalization experiment on the AMA/I_crane articulated object dataset. Unlike D-NeRF's synthetic rendered data, I_crane data originates from real-world capture scenarios, containing multi-view images (images/), camera calibration (calibration/), foreground silhouettes (silhouettes/), template mesh (template/), and pose data (poses/). We used the train_thinned/ directory with sparse training frames for model training.

RigGS's end-to-end training pipeline ran to completion without code-level crashes or CUDA runtime exceptions, demonstrating code-level compatibility with diverse input data formats. However, we observed clear anomalies: rendering output for some frames was invisible or severely blurred, with fuzzy edges and lost surface texture; the PSNR metric stabilized after several hundred iterations with minimal fluctuation and a flattened loss curve; and the active Gaussian count exhibited large training-time fluctuations, indicating instability in the adaptive density control mechanism.

Preliminary analysis suggests two contributing factors. First, data domain shift: I_crane's real-world image characteristics (lighting variation, self-shadows, motion blur) differ significantly from D-NeRF's synthetic data (uniform lighting, solid backgrounds). RigGS's Stage-1 MLP may encode data priors misaligned with I_crane's distribution. Second, COLMAP point cloud quality: real-world capture conditions degrade feature extraction and matching, reducing initial point cloud density and uniformity. Improvement directions include: training Stage-1 from scratch on I_crane data to eliminate domain shift; adjusting adaptive density control hyperparameters; and leveraging provided foreground silhouettes to exclude background gradient noise.

8. Discussion

Our test PSNR of 37.56 dB falls 3.26 dB below the original D-NeRF average. However, the original average includes easier scenes achieving >40 dB while hook is intrinsically harder due to rapid articulated motion. The ablation finding that skeleton modeling primarily improves perceptual quality (36.6% LPIPS reduction) aligns with qualitative observations in the original paper and provides a quantitative metric previously absent.

This study has limitations:

- full quantitative evaluation is limited to the hook scene due to computational constraints and our proposed code fixes have not been empirically validated through full retraining;
- the Unity system operates on pre-baked PLY exports rather than integrating a real-time 3DGS rasterizer;
- lightweight model compression encountered CUDA OOM on 24 GB, indicating optimization challenges for consumer deployment.

9. Conclusion

We have presented a comprehensive reproduction, ablation analysis, and extension study of RigGS, a CVPR 2025 template-free method for automatic skeleton rigging of articulated 3D Gaussians from monocular videos. Our full two-stage training on the D-NeRF hook scene achieves test PSNR 37.56 dB (SSIM 0.9945) with ablation experiments revealing a 36.6% LPIPS improvement from skeleton-driven modeling. We identify and document four critical implementation defects with precise localizations and fix proposals. Our Unity visualization system demonstrates that skeleton-driven 3DGS can be integrated into real-time interactive applications. All findings are verified against actual project files and training logs.

References

- [1] Y. Yao, Z. Deng, and J. Hou. RigGS: Rigging of 3D Gaussians for Modeling Articulated Objects in Videos. CVPR, 2025.
- [2] B. Kerbl, G. Kopanas, T. Leimkuhler, and G. Drettakis. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. ACM TOG, 42(4), 2023.
- [3] A. Pumarola, E. Corona, G. Pons-Moll, and F. Moreno-Noguer. D-NeRF: Neural Radiance Fields for Dynamic Scenes. CVPR, 2021.
- [4] B. Mildenhall et al. NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis. ECCV, 2020.
- [5] M. Loper et al. SMPL: A Skinned Multi-Person Linear Model. ACM TOG, 34(6), 2015.

- [6] S. Zuffi et al. 3D Menagerie: Modeling the 3D Shape and Pose of Animals. CVPR, 2017.
- [7] Z. Xu et al. RigNet: Neural Rigging for Articulated Characters. ACM TOG, 39(4), 2020.
- [8] L. Uzolas et al. Template-free Articulated Neural Point Clouds. NeurIPS, 2024.
- [9] G. Yang et al. BANMo: Building Animatable 3D Neural Models. CVPR, 2022.
- [10] Y.-H. Huang et al. SC-GS: Sparse-Controlled Gaussian Splatting. CVPR, 2024.
- [11] G. Wu et al. 4D Gaussian Splatting for Real-Time Dynamic Scene Rendering. CVPR, 2024.
- [12] J. Fang et al. Fast Dynamic Radiance Fields with Time-Aware Neural Voxels. SIGGRAPH Asia, 2022.
- [13] D. Wan et al. Template-free Articulated Gaussian Splatting. NeurIPS, 2024.
- [14] M. Caron et al. DINOv2: Learning Robust Visual Features without Supervision. arXiv:2304.07193, 2023.
- [15] T.Y. Zhang and C.Y. Suen. A Fast Parallel Algorithm for Thinning Digital Patterns. Commun. ACM, 27(3), 1984.
- [16] J.P. Lewis et al. Pose Space Deformation [16]. SIGGRAPH, 2000.
- [17] Y.-J. Yuan et al. NeRF-Editing: Geometry Editing of Neural Radiance Fields. CVPR, 2022.
- [18] Y. Chen et al. GaussianEditor: Swift and Controllable 3D Editing with Gaussian Splatting. CVPR, 2024.